

Manual Book Developer — UPFotoStudio

Dokumentasi teknis untuk developer yang mengembangkan & memelihara aplikasi **UPFotoStudio**.

Daftar Isi

- 1. Ringkasan & Tech Stack
- 2. Persyaratan Sistem
- 3. Instalasi & Setup Lokal
- 4. Konfigurasi Environment (.env)
- 5. Struktur Proyek
- 6. Arsitektur Aplikasi
- 7. Model & Skema Database
- 8. Routing
- 9. Autentikasi & Otorisasi
- 10. Alur Booking
- 11. Payment Gateway
- 12. Laporan & Ekspor
- 13. Email
- 14. Perintah Penting
- 15. Troubleshooting (Laragon/Windows)

1. Ringkasan & Tech Stack

UPFotoStudio adalah web app **Laravel 10** untuk booking sewa studio foto dengan pembayaran QRIS. Memiliki 3 peran: **tamu (frontend publik)**, **admin**, dan **owner**.

Komponen	Teknologi
Framework	Laravel 10
Bahasa	PHP 8.2 (Laragon) — <code>composer.json</code> minimum <code>^8.1</code>
Database	MySQL 8 (<code>db_upfotostudio</code>)
Frontend build	Vite + Bootstrap 5
PDF	<code>barryvdh/laravel-dompdf</code>
Excel	<code>maatwebsite/excel</code>
Auth helper	<code>laravel/sanctum</code>
Payment	Pluggable: <code>mock</code> / <code>midtrans</code> (Snap)

2. Persyaratan Sistem

- PHP 8.2+ dengan ekstensi standar Laravel (mbstring, openssl, pdo_mysql, gd, dll.)
- Composer
- Node.js + npm
- MySQL 8
- (Disarankan) Laragon di Windows

3. Instalasi & Setup Lokal

```
# 1. Dependencies PHP
#   (gunakan --ignore-platform-reqs karena composer.lock dibuat di PHP 8.4,
#   sedangkan environment lokal berjalan di PHP 8.2)
composer install --ignore-platform-reqs

# 2. Dependencies JS
npm install

# 3. Siapkan .env
copy .env.example .env
php artisan key:generate

# 4. Database
#   Pastikan MySQL berjalan, lalu buat DB `db_upfotostudio`.
#   Impor dump awal:
#   mysql -u root db_upfotostudio < db_upfotostudio.sql
php artisan migrate          # jalankan migrasi yang belum diterapkan

# 5. Build asset
npm run build                # atau: npm run dev (mode watch)

# 6. Jalankan server
php artisan serve --port=8080
#   → http://127.0.0.1:8080
```

Via Laragon vhost (opsional):

- Vhost: `C:\laragon\etc\apache2\sites-enabled\auto.upfotostudio.test.conf` → DocumentRoot ke `public/`
- `APP_URL=http://upfotostudio.test`
- Pastikan `upfotostudio.test` terdaftar di file `hosts` (restart Laragon agar otomatis ditambahkan).

Akun demo:

Peran	Email	Password
Admin	admin@upfoto.test	admin12345
Owner	owner@upfoto.test	owner12345

4. Konfigurasi Environment (.env)

Variabel penting (lihat `.env.example`):

```
APP_URL=http://upfotostudio.test

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_upfotostudio
DB_USERNAME=root
DB_PASSWORD=

# Payment gateway: mock | midtrans
PAYMENT_GATEWAY=mock

# Midtrans (jika PAYMENT_GATEWAY=midtrans)
MIDTRANS_MERCHANT_ID=
MIDTRANS_CLIENT_KEY=
MIDTRANS_SERVER_KEY=
MIDTRANS_IS_PRODUCTION=false
MIDTRANS_EXPIRY_MINUTES=30

# Mail (default Mailpit untuk dev)
MAIL_MAILER=smtp
MAIL_HOST=mailpit
MAIL_PORT=1025
```

5. Struktur Proyek

```
app/
├── Contracts/Payments/PaymentGatewayInterface.php # kontrak gateway
├── Http/Controllers/
│   ├── Auth/AuthController.php # login/logout admin & owner
│   ├── Frontend/ # publik: Home, Booking, Contact, PaymentRedirect
│   ├── Backend/Admin/ # Dashboard, Studio, ServicePackage, Booking,
│   │   # Transaction, Content, ContactMessage, Report
│   ├── Backend/Owner/ # Dashboard, Report
│   └── Api/ # MidtransNotificationController, PaymentCallbackController
├── Http/Requests/Frontend/StoreBookingRequest.php
├── Models/ # User, Guest, Studio, ServicePackage,
│   # Booking, PaymentTransaction, WebsiteContent, ContactMessage
├── Mail/ # BookingPendingMail, BookingConfirmedMail
├── Services/
│   ├── Bookings/BookingService.php # orkestrasi pembuatan booking (atomik)
│   ├── Payments/PaymentService.php # logika transaksi & notifikasi
│   └── Payments/MockQrisGateway.php # gateway simulasi lokal
```

```
└─ Payments/MidtransSnapGateway.php           # gateway Midtrans Snap (HTTP client)
└─ Reports/ReportService.php

routes/web.php                                # seluruh rute web + payment redirect
routes/api.php                                # webhook notifikasi
resources/views/
└─ frontend/      backend/admin  backend/owner
└─ layouts/        (frontend.blade.php, dashboard.blade.php)
└─ pdf/            (invoices, reports)
└─ emails/
docs/                                                    # dokumentasi (termasuk file ini)
```

6. Arsitektur Aplikasi

Pola yang digunakan: **Controller tipis + Service layer**.

- **Controller** menangani HTTP (validasi request, render view, redirect).
- **Service** memuat logika bisnis (booking, pembayaran, laporan).
- **Payment gateway** dipasang via **dependency injection** menggunakan interface, sehingga implementasi bisa ditukar lewat env tanpa mengubah pemanggil.

```
PaymentGatewayInterface ← di-bind di AppServiceProvider berdasarkan PAYMENT_GATEWAY
└─ MockQrisGateway      (PAYMENT_GATEWAY=mock)
└─ MidtransSnapGateway  (PAYMENT_GATEWAY=midtrans)
```

`BookingService` & `PaymentService` menerima `PaymentGatewayInterface` lewat constructor.

7. Model & Skema Database

Migrasi berada di `database/migrations/` (prefix `2026_03_*`).

Model	Tabel	Catatan penting
User	users	kolom <code>role</code> ditambahkan via migrasi terpisah (admin/owner)
Guest	guests	tamu pemesan; <code>firstOrCreate</code> by email+phone
Studio	studios	punya <code>is_active</code>
ServicePackage	service_packages	<code>price</code> , <code>duration_minutes</code> , <code>is_active</code> , <code>image_path</code>
Booking	bookings	inti pemesanan
PaymentTransaction	payment_transactions	1:1 dengan booking
WebsiteContent	website_contents	konten dinamis + <code>image_path</code>
ContactMessage	contact_messages	pesan dari form kontak

Konstanta `Booking`

```

STATUS_PENDING_PAYMENT = 'PENDING_PAYMENT'
STATUS_CONFIRMED        = 'CONFIRMED'
STATUS_CANCELLED        = 'CANCELLED'
STATUS_COMPLETED        = 'COMPLETED'

PAYMENT_DP      = 'DP'      // bayar 30% di muka
PAYMENT_LUNAS = 'LUNAS'    // bayar penuh

```

Relasi

```

Guest 1 — * Booking * — 1 Studio
      |
      * — 1 ServicePackage
      |
Booking 1 — 1 PaymentTransaction

```

- `Booking::guest()`, `studio()`, `servicePackage()` → `belongsTo`
- `Booking::paymentTransaction()` → `hasOne`

Identifier dibuat di `BookingService` :

- `booking_code` → `BOOK-<YmdHis>-<RAND4>`
- `invoice_number` → `INV-<YmdHis>-<RAND4>` (juga dipakai sebagai `order_id` Midtrans)

8. Routing

Semua rute web ada di `routes/web.php` .

Publik (frontend)

Method	URI	Name	Controller
GET	<code>/</code>	<code>frontend.home</code>	<code>HomeController@home</code>
GET	<code>/paket-harga</code>	<code>frontend.pricing</code>	<code>HomeController@pricing</code>
GET	<code>/booking/paket/{servicePackage}</code>	<code>frontend.booking.package-detail</code>	<code>BookingController@packageDetail</code>
GET	<code>/booking/order</code>	<code>frontend.booking.order</code>	<code>BookingController@order</code>
POST	<code>/booking</code>	<code>frontend.booking.store</code>	<code>BookingController@store</code>
GET	<code>/booking/status/{invoiceNumber}</code>	<code>frontend.booking.status</code>	<code>BookingController@status</code>
GET	<code>/booking/invoice/{invoiceNumber}</code>	<code>frontend.booking.invoice</code>	<code>BookingController@invoice</code>
GET	<code>/booking/paket/{servicePackage}/slots</code>	<code>frontend.booking.slots</code>	<code>BookingController@slotsJson</code> (AJAX)
GET	<code>/booking/studios/{studio}/packages</code>	<code>frontend.booking.packages-by-studio</code>	JSON helper

Method	URI	Name	Controller
POST	/kontak	frontend.contact.store	ContactMessageController@store

Payment redirect (didaftarkan di dashboard Midtrans)

/payment/finish , /payment/unfinish , /payment/error → PaymentRedirectController . finish melakukan sinkronisasi via Midtrans Status API agar localhost tetap berfungsi tanpa webhook.

Admin — prefix /admin , name admin. , middleware auth:admin + role:admin

- GET /admin/dashboard
- resource studios , resource service-packages , resource contents (kecuali show)
- GET /admin/bookings , PATCH /admin/bookings/{booking}/status
- GET /admin/transactions
- GET /admin/contact-messages , PATCH ../{contactMessage}/read
- GET /admin/reports , GET /admin/reports/export/pdf

Owner — prefix /owner , name owner. , middleware auth:owner + role:owner

- GET /owner/dashboard
- GET /owner/reports , GET /owner/reports/export/pdf , GET /owner/reports/export/excel

API — routes/api.php

- POST /api/payments/midtrans/notification → MidtransNotificationController (verifikasi signature sha512)

9. Autentikasi & Otorisasi

- Login gabungan untuk admin & owner di /login (AuthController@login).
- Menggunakan **guard terpisah**: auth:admin dan auth:owner ; group tamu pakai guest:admin,owner .
- Otorisasi peran lewat middleware role:admin / role:owner (berdasar kolom users.role).
- Logout terpisah: POST /admin/logout , POST /owner/logout .
- Frontend publik **tidak** memerlukan login.

10. Alur Booking

Diorkestrasi oleh BookingService::createBooking() — seluruh proses dibungkus DB::transaction (atomik).

1. **Lock & validasi** Studio dan ServicePackage (lockForUpdate , harus is_active).
2. **Hitung waktu**: end_time = start_time + duration_minutes .
3. **Validasi jadwal**:
 - Tidak boleh di masa lalu.
 - Harus dalam jam operasional **10:00-21:00** (sesi selesai sebelum tutup).
 - **Anti double-booking**: cek interval overlap pada studio yang sama untuk status PENDING_PAYMENT / CONFIRMED (lockForUpdate).

4. **Guest:** `firstOrCreate` berdasarkan email+phone; update nama bila berubah.
5. **Hitung biaya:** `total_amount = price + add_on_amount`. Untuk DP, `amount = round(total * 0.3)`; LUNAS = total.
6. **Buat `Booking`** (status `PENDING_PAYMENT`) dan **`PaymentTransaction`** (status `PENDING`, method `QRIS`).
7. **Panggil gateway** `createQrisPayment()` → simpan `gateway_reference`, `qr_payload`, `expires_at` (default +30 menit).
8. **Kirim email** instruksi pembayaran via `DB::afterCommit` (kegagalan email di-log, tidak menggagalkan booking).

Perhitungan slot (`BookingController::buildAvailableSlots`)

- Slot dibuat tiap **30 menit** dari 10:00 sampai sebelum 21:00.
- Sebuah slot `available` jika: sesi selesai $\leq$ jam tutup, **dan** belum lewat, **dan** tidak overlap dengan booking aktif.
- Endpoint AJAX: `GET /booking/paket/{servicePackage}/slots?date=YYYY-MM-DD`.

Add-on

Katalog di-hardcode di `BookingController::getAddOnCatalog()` (Extra Print, Jas Hitam, Costume, Extra Time, Keychain). Add-on, pilihan background, dan consent media sosial digabung ke kolom `notes`.

Catatan: menambah add-on baru cukup menambah entri pada `getAddOnCatalog()`. Pilihan dari user divalidasi ulang terhadap katalog di `resolveSelectedAddOns()` (qty $\geq$ 1).

11. Payment Gateway

Pluggable melalui env `PAYMENT_GATEWAY`:

Nilai	Implementasi	Untuk
<code>mock</code>	<code>MockQrisGateway</code>	Pengembangan lokal tanpa kredensial
<code>midtrans</code>	<code>MidtransSnapGateway</code>	Sandbox / production Midtrans

- Binding interface `PaymentGatewayInterface` dilakukan di `AppServiceProvider`.
- Midtrans **Snap** diintegrasikan memakai **Laravel HTTP client** (tanpa SDK resmi).
- `invoice_number` = Midtrans `order_id`.
- **Webhook:** `POST /api/payments/midtrans/notification` (signature sha512 diverifikasi) → `PaymentService::processMidtransNotification`.
- **Redirect:** `/payment/{finish,unfinish,error}`. `finish` menyinkronkan status via Midtrans Status API sehingga localhost tetap berfungsi tanpa webhook publik.
- Dokumentasi setup detail: `docs/MIDTRANS_SETUP.md`.

Kontrak gateway

`PaymentGatewayInterface::createQrisPayment(array $payload)` menerima `invoice_number`, `amount`, `customer_*`, `booking_code` dan mengembalikan array berisi `reference`, `qr_string`, `payment_url`, `expires_at`.

12. Laporan & Ekspor

- `ReportService` menyiapkan agregasi data laporan.
- **Admin**: ekspor **PDF** (DomPDF, view `resources/views/pdf/reports/`).
- **Owner**: ekspor **PDF** dan **Excel** (Maatwebsite/Excel).
- Invoice PDF tamu: `BookingController::invoice()` → view `pdf/invoices/invoice` , ukuran A4 portrait.

13. Email

- `BookingPendingMail` — instruksi pembayaran, dikirim setelah booking dibuat (via `afterCommit`).
- `BookingConfirmedMail` — konfirmasi setelah pembayaran sukses.
- View: `resources/views/emails/booking-pending.blade.php` , `booking-confirmed.blade.php` .
- Dev: gunakan **Mailpit** (`MAIL_HOST=mailpit` , port 1025) untuk menangkap email.

14. Perintah Penting

```
# Server dev
php artisan serve --port=8080

# Migrasi
php artisan migrate
php artisan migrate:status
php artisan migrate:fresh --seed      # reset (HATI-HATI: menghapus data)

# Cache & config
php artisan optimize:clear
php artisan config:clear
php artisan route:list

# Asset
npm run dev      # watch
npm run build    # produksi

# Tinker
php artisan tinker
```

15. Troubleshooting (Laragon/Windows)

Masalah	Solusi
<code>composer install</code> gagal karena versi PHP	Tambah flag <code>--ignore-platform-reqs</code> (lock dibuat di PHP 8.4, runtime 8.2).
Laravel error "could not connect to database"	Pastikan MySQL berjalan: <code>Start-Process mysql</code> . Cek <code>DB_DATABASE=db_upfotostudio</code> .

Masalah	Solusi
<code>upfotostudio.test</code> tidak bisa dibuka	Restart Laragon atau tambahkan host manual di <code>C:\Windows\System32\drivers\etc\hosts</code> .
Halaman blank / asset hilang	Jalankan <code>npm run build</code> lalu <code>php artisan optimize:clear</code> .
Perubahan rute/config tidak terbaca	<code>php artisan config:clear && php artisan route:clear</code> .
Webhook Midtrans tak masuk di localhost	Andalkan redirect <code>/payment/finish</code> (sinkron via Status API), atau gunakan tunnel publik (mis. ngrok).
Email tidak terkirim	Cek konfigurasi <code>MAIL_*</code> ; untuk dev jalankan Mailpit. Kegagalan email di-log, tidak menggagalkan booking.

Lokasi penting (lingkungan lokal user)

- Proyek: `c:\laragon\www\UPFotoStudio\UPFotoStudio\`
- MySQL binary: `C:\laragon\bin\mysql\mysql-8.0.30-winx64\bin\`

Untuk panduan penggunaan non-teknis, lihat `docs/MANUAL_PENGGUNA.md`.